Практична робота № 1

Тема: Базові конструкції, структури даних та конкурентність у Go

---

Мета роботи

- Набути практичних навичок роботи з системою типів мови Go: структурами (`struct`), методами з одержувачами (`receiver`), явним опрацюванням помилок (`error`).
- Опанувати принципи проектування інтерфейсів (`interface`) та неявної імплементації (качиної типізації).
- Засвоїти фундаментальну парадигму конкурентності CSP: запуск легковажних потоків (`goroutine`), синхронізацію їхньої роботи через лічильники `sync.WaitGroup` та взаємне виключення `sync.Mutex`.
- Опанувати потокобезпечний обмін даними між горутинами за допомогою каналів (`chan`) та їхнє закриття.

Технічне забезпечення

- Операційна система: Windows, macOS або Linux.
- Середовище розробки: Visual Studio Code (з офіційним розширенням Go) або JetBrains GoLand.
- Компілятор: Go SDK версії 1.22 або новішої.
- Термінал / CLI: bash, zsh або PowerShell.
- Система контролю версій: Git.
- Утиліти діагностики: вбудований детектор стану гонитви пам’яті (`go run -race`).

Короткі теоретичні відомості

1. Структури, методи та неявна реалізація інтерфейсів

У Go відсутній механізм класичного успадкування класів (`class … extends`). Замість цього використовуються структури даних (`struct`) та методи, прив'язані до конкретного типу через одержувач (**receiver**).

Інтерфейс у Go визначає контракт поведінки — перелік сигнатур методів. Якщо тип містить реалізацію всіх методів інтерфейсу, він вважається таким, що задовольняє цей інтерфейс **автоматично та неявно** (принцип качиної типізації: *«якщо щось ходить як качка і крякає як качка — це качка»*). Жодних ключових слів на зразок `implements` не існує.

```go
// Оголошення інтерфейсу
type Greeter interface {
    Greet(name string) string ①
}

// Структура
type Bot struct {
    Prefix string
}

// Метод структури з value receiver, що неявно імплементує інтерфейс Greeter
func (b Bot) Greet(name string) string { ②
    return b.Prefix + ", " + name + "!"
}
```

① Оголошення контракту поведінки без прив'язки до конкретної реалізації.

② Неявна імплементація інтерфейсу через збіг сигнатури методу.

---

2. Горутини та синхронізація через `sync.WaitGroup`

Горутина (`goroutine`) — це легковажний потік виконання, яким керує не ядро операційної системи, а вбудований планувальник середовища Go (GMP-scheduler). Початковий розмір стека горутини становить лише близько 2–4 КБ, що дозволяє запускати тисячі таких потоків паралельно без відчутних накладних витрат.

Для очікування завершення групи горутин використовується структура `sync.WaitGroup` з трьома основними методами:

- `Add(n)` — збільшує внутрішній лічильник активних задач на `n` (викликається **до** запуску горутини);

- `Done()` — зменшує лічильник на 1 (зазвичай викликається через `defer` всередині самої горутини);
- `Wait()` — блокує поточну горутину до моменту, поки лічильник не обнулиться.

```go
var wg sync.WaitGroup

for i := 1; i <= 3; i++ {
    wg.Add(1) ①
    go func(id int) {
        defer wg.Done() ②
        // Виконання корисної роботи горутини
    }(i) ③
}

wg.Wait() ④
```

① Збільшення лічильника обов'язково виконується до запуску горутини в головному потоці.

② Гарантоване декрементування лічильника при виході з функції горутини.

③ Передача змінної циклу як аргументу для запобігання стану гонитви на замиканні.

④ Блокування до моменту, поки лічильник WaitGroup не дорівнюватиме 0.

---

3. Канали (chan) як безпечний транспорт даних

Парадигма конкурентності Go (CSP — Communicating Sequential Processes) спирається на принцип:

> *«Не спілкуйтеся, розділяючи пам'ять; натомість діліться пам'яттю, спілкуючись».*

Канал — це типізований потокобезпечний конвеєр для передачі даних між горутинами:

- `ch := make(chan string)` — небуферизований канал (блокує відправника, поки отримувач не буде готовий прийняти дані);
- `ch := make(chan string, 10)` — буферизований канал (блокує лише у разі заповнення буфера);
- `ch <- value` — надсилання значення в канал;
- `val := <-ch` — вичитування значення з каналу;
- `close(ch)` — закриття каналу відправником після закінчення надсилання даних;
- `for item := range ch` — послідовне вичитування всіх елементів до моменту закриття каналу.

```go
results := make(chan int, 5) ①

// Відправник
go func() {
    for i := 1; i <= 5; i++ {
        results <- i * 10 ②
    }
    close(results) ③
}()

// Отримувач (вичитує до закриття каналу)
for val := range results { ④
    _ = val
}
```

① Ініціалізація буферизованого каналу місткістю на 5 елементів.

② Запис результату обчислення у канал.

③ Закриття каналу сигналізує отримувачу про відсутність нових даних.

④ Цикл автоматично завершується, коли канал спустошено та закрито.

---

4. Вимірювання часу виконання

Для заміру швидкодії операцій використовується стандартний пакет `time`:

- `time.Now()` фіксує поточний момент часу;
- `time.Since(start)` повертає тривалість (`time.Duration`), що минула від зафіксованої мітки часу.

Постановка задачі

У клієнт-серверних архітектурах сервіси регулярно опитують зовнішні або суміжні API для перевірки їхньої працездатності (Health Check). Якщо робити це послідовно в одному потоці, опитування сотень сервісів займе неприпустимо багато часу.

Необхідно розробити консольну утиліту для **моніторингу доступності мережевих ресурсів (HTTP URL Health Checker)**, яка:

1. Зберігає перелік цільових адрес та виконує опитування їхнього стану;
2. Фіксує результати кожного запиту: код відповіді (наприклад, 200 OK), час очікування та можливі помилки з’єднання;
3. Демонструє еволюцію архітектурного рішення: від базової послідовної моделі до конкурентної обробки через горутини, синхронізацію та канали зв’язку.

Завдання

Практична робота складається з трьох послідовних рівнів. Кожен наступний рівень базується на попередньому та розширює функціональність розроблюваного рішення.

Рівень 1. Базовий: структури, методи та послідовний моніторинг

1. **Ініціалізація проєкту:**
   - Створити окрему робочу директорію для лабораторної роботи.
   - Ініціалізувати Go-модуль за допомогою команди `go mod init <назва_модуля>`.
   - Створити файл `main.go`.
2. **Моделювання результатів:**
   - Описати структуру для збереження результату опитування адреси:
     - URL-адреса ресурсу (`string`);
     - HTTP-статус код відповіді (`int`);
     - тривалість виконання запиту (`time.Duration`);
     - помилка у разі невдалого з'єднання (`error`).
3. **Проєктування інтерфейсу та обробника:**
   - Оголосити інтерфейс для абстрагування операції перевірки ресурсу (контракт має приймати рядок URL та повертати структуру результату).
   - Створити структуру HTTP-клієнта з конфігурацією таймауту або внутрішнім полем клієнта.
   - Реалізувати для цієї структури метод перевірки адреси (використати стандартний пакет `net/http` та функцію `http.Get` або метод `Do` власного клієнта `http.Client`).
   - Усередині методу забезпечити точний замір тривалості виконання запиту за допомогою `time.Now()` та `time.Since()`.
4. **Послідовний прогін:**
   - У функції `main()` сформувати зріз (`slice`) із 4–5 різних адрес (включно з 2–3 реальними сайтами, наприклад `https://go.dev`, `https://github.com`, а також завідомо некоректним або неіснуючим доменом для тестування відловлювання помилок).
   - У послідовному циклі опитати всі URL через інтерфейс та вивести у термінал статус, тривалість і текст помилки (якщо є).

Рівень 2. Конкурентний: горутини та захист спільного стану через Mutex

1. **Збереження функціональності Рівня 1.**
2. **Розпаралелювання запитів:**
   - Замість блокуючого послідовного циклу організувати запуск перевірки кожного URL в окремій легковажній горутині за допомогою ключового слова `go`.
   - Звернути особливу увагу на коректну передачу поточної адреси всередину анонімної функції горутини (уникнення замикання змінної циклу).
3. **Координація завершення горутин:**
   - Використати примітив синхронізації `sync.WaitGroup`, щоб головний потік програми дочекався закінчення опитування всіх серверів і не завершився завчасно.
   - Забезпечити коректне збільшення лічильника перед запуском кожної горутини та виклик зменшення лічильника всередині горутини (бажано через `defer`).
4. **Збір агрегованої статистики та ліквідація стану гонитви:**
   - Оголосити спільну структуру для підрахунку підсумкової статистики (наприклад: загальна кількість перевірених адрес, кількість успішних відповідей зі статусом 200, кількість помилок).
   - Оскільки горутини модифікують спільні лічильники одночасно, забезпечити захист критичної секції за допомогою `sync.Mutex` (методи `Lock()` та `Unlock()`).
   - Перевірити програму на відсутність стану гонитви пам'яті за допомогою прапорця:

     ```
     1 go run -race .
     ```

     Консоль не повинна містити жодних попереджень WARNING: DATA RACE.

Рівень 3. Просунутий: асинхронний збір результатів через канали

1. **Збереження функціональності попередніх рівнів.**
2. **Перехід до передачі даних через канали:**
   - Замість безпосереднього оновлення спільних лічильників через м'ютекс створити канал для передачі структур результатів із горутин.
   - Після виконання перевірки горутина повинна відправити отриману структуру результату в цей канал.
3. **Організація життєвого циклу каналу:**
   - Запустити окрему допоміжну горутину-координатор, яка очікує завершення всіх обчислень через `sync.WaitGroup.Wait()` і одразу після цього безпечно закриває канал

результатів за допомогою функції `close()`.

4. **Обробка потоку результатів у головному потоці:**
   - У функції `main()` організувати вичитування всіх результатів із закритого каналу за допомогою конструкції `for res := range resultsChan`.
   - Здійснити підрахунок успішних/неуспішних результатів та вивести підсумковий структурований звіт моніторингу.
5. **Порівняльний аналіз продуктивності:**
   - Виміряти та вивести сумарний час роботи послідовного опитування (з Рівня 1) та паралельного опитування (з Рівня 3).
   - Зафіксувати кратне прискорення виконання завдання при конкурентному підході.

Приклад очікуваного виведення програми

```
=== HTTP URL Health Checker ===

[Конкурентна перевірка адрес...]
-> https://go.dev: Status 200 OK (час: 124ms)
-> https://github.com: Status 200 OK (час: 215ms)
-> https://httpbin.org/status/404: Status 404 Not Found (час: 310ms)
-> https://invalid-fake-host-test.org: Помилка (Get "https://invalid-fake-host-
test.org": dial tcp: lookup: no such host)

========================================
ПІДСУМКОВИЙ ЗВІТ МОНІТОРИНГУ:
- Усього перевірено адрес: 4
- Успішних з'єднань: 3
- Мережевих помилок: 1
- Загальний час роботи: 320ms (проти 980ms у послідовному режимі)
========================================
```

Контрольні питання

1. Чим відрізняється неявна реалізація інтерфейсів у мові Go від явного контракту (`implements`) у таких мовах, як Java чи C#?
2. Яка різниця між передачею одержувача методу за значенням (`value receiver`) та за вказівником (`pointer receiver`)?
3. Чому запуск 1 000 горутин у Go не призводить до вичерпання пам'яті комп'ютера, тоді як запуск такої ж кількості системних потоків ОС у Java чи C++ може спричинити збій?

---

4. Що таке стан гонитви (race condition)? Як прапорець `-race` компілятора Go допомагає його виявляти?
5. Для чого призначена структура `sync.WaitGroup`? Чому виклик `Add()` обов'язково має передувати оператору `go`?
6. У чому полягає відмінність між блокуванням спільної пам'яті за допомогою `sync.Mutex` та передачею володіння даними через `chan`?
7. Що трапиться, якщо спробувати надіслати дані у вже закритий канал або закрити вже закритий канал повторно?

Критерії оцінювання

Оцінювання здійснюється за шкалою повноти реалізації рівнів практичної роботи:

- **«Задовільно» (Рівень 1):** реалізовано базовий прототип, створено структуру проєкту, програма успішно компілюється та виконує базові сценарії.
- **«Добре» (Рівень 2):** виконано базовий рівень та реалізовано ключову функціональність заняття, забезпечено надійність і безпеку виконання коду.
- **«Відмінно» (Рівень 3):** повністю завершено всі три рівні завдання з дотриманням кращих інженерних практик, чистоти архітектури та оптимізації.
- **Усний захист:** обов'язкове пояснення логіки коду та впевнені відповіді на контрольні питання викладача.

Вимоги до звітності

1. Опублікувати вихідний код у Git-репозиторії.
2. Оформити файл `README.md` з описом проєкту, інструкцією запуску та обов'язковими скриншотами роботи програми (включаючи демонстрацію відсутності помилок).
3. Продемонструвати роботу програми викладачеві та захистити результати.